UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	M. Rizki Algipari / 2428240069
Kelas	SI5B
Judul project / topik API	Topik 14 - Galeri Seni: Lukisan (RESTful API /paintings)
Tautan repositori	https://github.com/Ridzz05/si2b_kuis1_mrizkialgipari
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	19 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner
Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

API yang diuji adalah RESTful API Topik 14 Galeri Seni dengan entitas utama lukisan pada resource /paintings. Project dibangun memakai Express.js 5 dengan pemisahan Route, Controller, Model, dan Middleware, dan menyimpan data pada array di memori. Setiap operasi tulis dilindungi header x-api-key yang diperiksa middleware cekApiKey.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/	Tidak	200	Informasi API (identitas, topik, daftar endpoint)
2	GET	/paintings	Tidak	200	Ambil semua data lukisan (mendukung ?aliran=)
3	GET	/paintings/:id	Tidak	200	Ambil satu data lukisan
4	POST	/paintings	Ya	201	Tambah data lukisan
5	PUT	/paintings/:id	Ya	200	Ubah data lukisan
6	DELETE	/paintings/:id	Ya	204	Hapus data lukisan
7	GET	/paintings?aliran=realisme	Tidak	200	Filter lukisan berdasarkan aliran

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

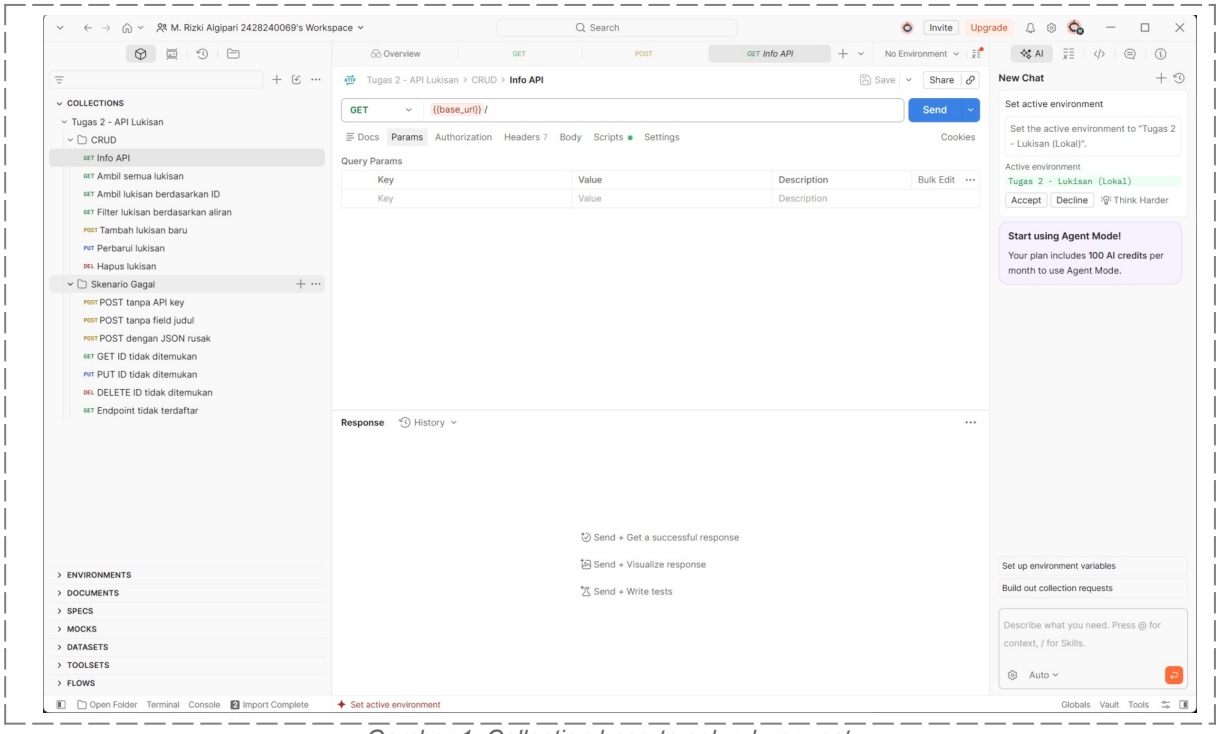
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server
2	api_key	(nilai API_KEY lokal dari .env)	Nilai header kunci API
3	paintingId	(kosong, diisi script)	ID dari respons POST

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Request saya kelompokkan ke dua folder: CRUD untuk alur normal dan Skenario Gagal untuk kasus penolakan. Folder CRUD saya urutkan GET, POST, PUT, lalu DELETE karena request PUT dan DELETE memakai paintingId yang baru diisi dari respons POST. Folder Skenario Gagal memakai ID literal 999999 supaya tidak mengganggu urutan itu.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	Info API	GET	{{base_url}}/	CRUD
2	Ambil semua lukisan	GET	{{base_url}}/paintings	CRUD
3	Ambil lukisan berdasarkan ID	GET	{{base_url}}/paintings/1	CRUD
4	Filter lukisan berdasarkan aliran	GET	{{base_url}}/paintings?aliran=realisme	CRUD

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
5	Tambah lukisan baru	POST	{{base_url}}/paintings	CRUD
6	Perbarui lukisan	PUT	{{base_url}}/paintings/{{paintingId}}	CRUD
7	Hapus lukisan	DELETE	{{base_url}}/paintings/{{paintingId}}	CRUD
8	POST tanpa API key	POST	{{base_url}}/paintings	Skenario Gagal
9	POST tanpa field judul	POST	{{base_url}}/paintings	Skenario Gagal
10	POST dengan JSON rusak	POST	{{base_url}}/paintings	Skenario Gagal
11	GET ID tidak ditemukan	GET	{{base_url}}/paintings/999999	Skenario Gagal
12	PUT ID tidak ditemukan	PUT	{{base_url}}/paintings/999999	Skenario Gagal
13	DELETE ID tidak ditemukan	DELETE	{{base_url}}/paintings/999999	Skenario Gagal
14	Endpoint tidak terdaftar	GET	{{base_url}}/tidak-ada	Skenario Gagal

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	Info API	Status 200 OK	Respons berupa objek; identitas dan resource sesuai
2	Ambil semua lukisan	Status 200 OK	Respons array; minimal 3 lukisan; setiap lukisan punya field wajib
3	Ambil lukisan berdasarkan ID	Status 200 OK	Lukisan id 1 ditemukan; judul sesuai data awal
4	Filter lukisan berdasarkan aliran	Status 200 OK	Respons array; seluruh hasil beraliran realisme
5	Tambah lukisan baru	Status 201 Created	status success; data.id angka; judul dan pelukis sesuai; paintingId disimpan
6	Perbarui lukisan	Status 200 OK	status success; id sama dengan paintingId; judul dan harga berubah; pesan menyebut id
7	Hapus lukisan	Status 204 No Content	Respons kosong
8	POST tanpa API key	Status 401 Unauthorized	Pesan "API key tidak valid atau tidak ditemukan"
9	POST tanpa field judul	Status 400 Bad Request	Pesan "Field judul wajib diisi"
10	POST dengan JSON rusak	Status 400 Bad Request	Pesan "Format JSON tidak valid"
11	GET ID tidak ditemukan	Status 404 Not Found	Pesan menyebut id 999999
12	PUT ID tidak ditemukan	Status 404 Not Found	Pesan menyebut id 999999
13	DELETE ID tidak ditemukan	Status 404 Not Found	Pesan menyebut id 999999

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
14	Endpoint tidak terdaftar	Status 404 Not Found	Pesan "Endpoint tidak ditemukan"

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
pm.test("Status 200 OK", () => pm.response.to.have.status(200));
pm.test("Respons berupa array", () => {
  const b = pm.response.json();
  pm.expect(b).to.be.an("array");
});
pm.test("Minimal berisi tiga lukisan awal", () => {
  const b = pm.response.json();
  pm.expect(b.length).to.be.at.least(3);
});
pm.test("Setiap lukisan memiliki field wajib", () => {
  const b = pm.response.json();
  b.forEach((lukisan) => {
    pm.expect(lukisan).to.have.all.keys("id", "judul", "pelukis", "aliran", "tahunDibuat",
"harga");
  });
});
```

Pada request Ambil semua lukisan, pm.test pertama memeriksa status 200 supaya request dianggap berhasil hanya bila server membalas OK. Tiga pm.test berikutnya memeriksa bentuk data: respons harus array, panjangnya minimal tiga karena data awal berisi tiga lukisan, dan setiap objek wajib memuat field id, judul, pelukis, aliran, tahunDibuat, dan harga. Pemeriksaan field ini penting karena perubahan pada model bisa menghilangkan field tanpa mengubah status code.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
pm.test("Status 201 Created", () => pm.response.to.have.status(201));
pm.test("Status respons bernilai success", () => {
  const b = pm.response.json();
  pm.expect(b.status).to.eql("success");
  pm.expect(b.message).to.eql("Data berhasil ditambahkan");
});
pm.test("Data lukisan baru memiliki id berupa angka", () => {
  const b = pm.response.json();
  pm.expect(b.data).to.be.an("object");
  pm.expect(b.data.id).to.be.a("number");
});
pm.test("Judul dan pelukis lukisan baru sesuai", () => {
  const b = pm.response.json();
  pm.expect(b.data.judul).to.eql("Lukisan Uji Otomasi");
  pm.expect(b.data.pelukis).to.eql("M. Rizki Algipari");
});
if (pm.response.code === 201) {
  pm.environment.set("paintingId", pm.response.json().data.id);
}
```

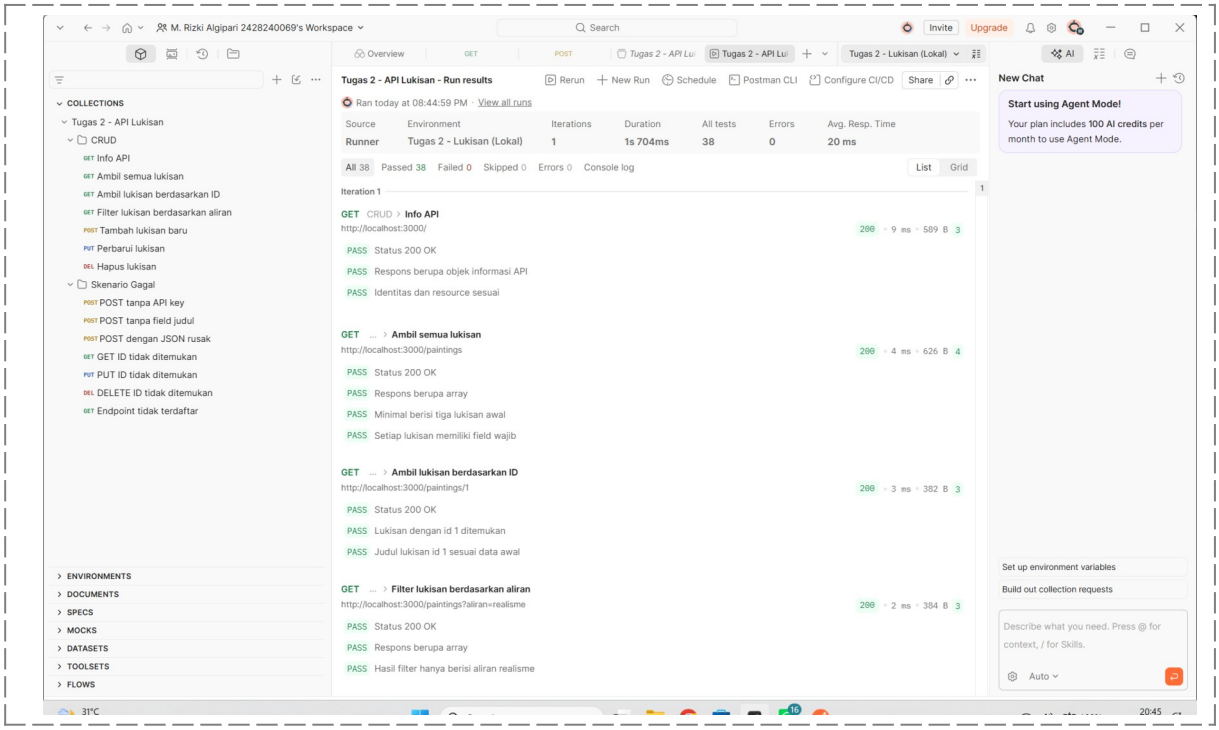
Request Tambah lukisan baru memeriksa status 201, nilai status dan message, tipe data id, serta kecocokan judul dan pelukis. Setelah lolos pemeriksaan, baris pm.environment.set("paintingId", ...) menyimpan id lukisan baru ke Environment. Request Perbarui lukisan dan Hapus lukisan memakai {{paintingId}} tersebut, jadi bila urutan ditukar dan PUT dijalankan sebelum POST, variabel masih kosong dan alamatnya menjadi /paintings/ yang dibalas 404.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Seluruh 14 request dijalankan sekaligus dan 38 assertion lulus tanpa kegagalan. Sebelum menjalankan Runner, saya menyalakan ulang server supaya array lukisan kembali ke tiga data awal dan paintingId pada Environment masih kosong. Environment yang dipakai adalah Tugas2_Lukisan.

4.2 Rekap angka hasil

Nama Collection	Tugas 2 - API Lukisan
Environment yang dipakai	Tugas2_Lukisan
Jumlah request dijalankan	14
Jumlah test lulus	38
Jumlah test gagal	0
Tanggal dan jam pengujian	19 Oktober 2026

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	Seluruh request	tidak ada kegagalan	Seluruh assertion lulus pada run pertama	-

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

```
$ npx newman run postman/Tugas2_Lukisan.postman_collection.json -e postman/Tugas2_Lukisan.postman_environment.json --reporters cli
newman

Tugas 2 - API Lukisan

□ CRUD
└ Info API
  GET http://localhost:3000/ [200 OK, 589B, 34ms]
  √  Status 200 OK
  √  Respons berupa objek informasi API
  √  Identitas dan resource sesuai

└ Ambil semua lukisan
  GET http://localhost:3000/paintings [200 OK, 626B, 4ms]
  √  Status 200 OK
  √  Respons berupa array
  √  Minimal berisi tiga lukisan awal
  √  Setiap lukisan memiliki field wajib

└ Ambil lukisan berdasarkan ID
  GET http://localhost:3000/paintings/1 [200 OK, 382B, 3ms]
  √  Status 200 OK
  √  Lukisan dengan id 1 ditemukan
  √  Judul lukisan id 1 sesuai data awal

└ Filter lukisan berdasarkan aliran
  GET http://localhost:3000/paintings?aliran=realisme [200 OK, 384B, 3ms]
  √  Status 200 OK
  √  Respons berupa array
  √  Hasil filter hanya berisi aliran realisme

└ Tambah lukisan baru
  POST http://localhost:3000/paintings [201 Created, 463B, 3ms]
  √  Status 201 Created
  √  Status respons bernilai success
  √  Data lukisan baru memiliki id berupa angka
  √  Judul dan pelukis lukisan baru sesuai

└ Perbarui lukisan
  PUT http://localhost:3000/paintings/10 [200 OK, 496B, 3ms]
  √  Status 200 OK
  √  Status respons bernilai success
  √  Lukisan yang diperbarui adalah paintingId yang disimpan
  √  Judul dan harga lukisan berhasil diperbarui
  √  Pesan menyebut id lukisan yang diperbarui

└ Hapus lukisan
  DELETE http://localhost:3000/paintings/10 [204 No Content, 166B, 2ms]
  √  Status 204 No Content
  √  Respons kosong

□ Skenario Gagal
└ POST tanpa API key
  POST http://localhost:3000/paintings [401 Unauthorized, 360B, 2ms]
  √  Status 401 Unauthorized
  √  Pesan API key tidak valid atau tidak ditemukan

└ POST tanpa field judul
  POST http://localhost:3000/paintings [400 Bad Request, 342B, 3ms]
  √  Status 400 Bad Request
  √  Pesan field judul wajib diisi

└ POST dengan JSON rusak
  POST http://localhost:3000/paintings [400 Bad Request, 342B, 3ms]
  √  Status 400 Bad Request
  √  Pesan format JSON tidak valid

└ GET ID tidak ditemukan
  GET http://localhost:3000/paintings/999999 [404 Not Found, 354B, 2ms]
  √  Status 404 Not Found
  √  Pesan data id 999999 tidak ditemukan

└ PUT ID tidak ditemukan
  PUT http://localhost:3000/paintings/999999 [404 Not Found, 354B, 2ms]
  √  Status 404 Not Found
  √  Pesan data id 999999 tidak ditemukan

└ DELETE ID tidak ditemukan
  DELETE http://localhost:3000/paintings/999999 [404 Not Found, 354B, 1ms]
  √  Status 404 Not Found
  √  Pesan data id 999999 tidak ditemukan

└ Endpoint tidak terdaftar
  GET http://localhost:3000/tidak-ada [404 Not Found, 341B, 2ms]
  √  Status 404 Not Found
  √  Pesan endpoint tidak ditemukan

┌─────────────────────────┬──────────┬──────────┐
│                         │ executed │   failed │
├─────────────────────────┼──────────┼──────────┤
│              iterations │        1 │        0 │
├─────────────────────────┼──────────┼──────────┤
│                requests │       14 │        0 │
├─────────────────────────┼──────────┼──────────┤
│            test-scripts │       14 │        0 │
├─────────────────────────┼──────────┼──────────┤
│      prerequest-scripts │        0 │        0 │
├─────────────────────────┼──────────┼──────────┤
│              assertions │       38 │        0 │
├─────────────────────────┴──────────┴──────────┤
│ total run duration: 768ms                     │
├───────────────────────────────────────────────┤
│ total data received: 1.85kB (approx)          │
├───────────────────────────────────────────────┤
│ average response time: 4ms [min: 1ms, max: 34ms, s.d.: 8ms] │
└───────────────────────────────────────────────┘
```

Gambar 3. Hasil menjalankan Collection dengan Newman

Gambar 3. Hasil menjalankan Collection dengan Newman

Perintah yang saya jalankan: npx newman run postman/Tugas2_Lukisan.postman_collection.json -e postman/Tugas2_Lukisan.postman_environment.json --reporters cli. Newman keluar dengan exit code 0, yang berarti seluruh assertion lulus. Exit code 1 akan muncul bila ada satu saja assertion gagal, sehingga kode ini bisa dipakai sebagai syarat otomatis pada pipeline.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/Ridzz05/si2b_kuis1_mrizkialgipari
Lokasi berkas Collection (.json)	postman/Tugas2_Lukisan.postman_collection.json
Lokasi berkas Environment (.json)	postman/Tugas2_Lukisan.postman_environment.json
Berkas Environment tidak memuat kunci rahasia sungguhan	[x] Ya [ ] Tidak
Repositori dapat dibuka dosen	[x] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Assertion yang paling berguna bagi saya adalah pemeriksaan field wajib pada request Ambil semua lukisan, karena pemeriksaan itu langsung menunjuk objek yang bentuknya menyimpang dari model. Assertion status code saja tidak cukup: respons 200 tetap lolos walau field tahunDibuat hilang dari data.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Menguji manual berarti saya harus membuka satu per satu request dan memeriksa responsnya dengan mata, sedangkan Collection Runner menjalankan semua request berurutan dan melaporkan assertion yang gagal dalam satu tabel. Bagian tersulit saat menulis test script adalah menjaga urutan request yang bergantung pada paintingId, dan saya atasi dengan menyimpan id hanya ketika status 201 serta memakai ID literal 999999 pada skenario gagal.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	✓
2	Setiap endpoint API saya ada di Collection	✓
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	✓
4	Setiap request punya assertion status code dan struktur JSON	✓
5	Nama setiap test berupa kalimat yang menyatakan harapan	✓
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	✓
7	Server dinyalakan ulang sebelum Runner dijalankan	✓
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	✓
9	Berkas Environment tidak memuat kunci rahasia sungguhan	✓
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	✓

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen